LAB 1

1. Write a C program to simulate SJF scheduling algorithm non pre-emptive.

```
#include <stdio.h>
#define MAX 10

struct process {
   int at, bt, ct, wt, tat, completed;
};

void input(struct process p[], int n) {
   for(int i=0; i<n; i++) {
      printf("Enter arrival time and burst time of process %d: ", i+1);
      scanf("%d%d", &p[i].at, &p[i].bt);
      p[i].completed = 0;
   }
}

void sjf(struct process p[], int n) {
   int time = 0, completed = 0;
   float avgTat = 0, avgWt = 0;

   while(completed < n) {
      int idx = -1;
      int minBt = 9999
      ;

      for(int i=0; i<n; i++) {
         if(p[i].at <= time && !p[i].completed) {
            if(p[i].bt < minBt) {
               minBt = p[i].bt;
```

```
                idx = i;
            }
        }
    }

    if(idx == -1) {
        time++;
        continue;
    }

    time += p[idx].bt;
    p[idx].ct = time;
    p[idx].tat = p[idx].ct - p[idx].at;
    p[idx].wt = p[idx].tat - p[idx].bt;
    p[idx].completed = 1;
    completed++;

    printf("Process %d -> CT: %d, TAT: %d, WT: %d\n", idx+1, p[idx].ct, p[idx].tat, p[idx].wt);
  }

  for(int i=0; i<n; i++) {
    avgTat += p[i].tat;
    avgWt += p[i].wt;
  }

  printf("\nAverage TAT: %.2f ms\n", avgTat/n);
  printf("Average WT: %.2f ms\n", avgWt/n);
}

int main() {
  struct process p[MAX];
  int n;
```

```
    printf("Enter number of processes: ");
    scanf("%d", &n);
    input(p, n);
    sjf(p, n);
    return 0;
}
```

Output:

```
Enter number of processes: 5
Enter arrival time and burst time of process 1: 0
3
Enter arrival time and burst time of process 2: 2

2
Enter arrival time and burst time of process 3: 3
5
Enter arrival time and burst time of process 4: 4
4
Enter arrival time and burst time of process 5: 6
1
Process 1 -> CT: 3, TAT: 3, WT: 0
Process 2 -> CT: 5, TAT: 3, WT: 1
Process 4 -> CT: 9, TAT: 5, WT: 1
Process 5 -> CT: 10, TAT: 4, WT: 3
Process 3 -> CT: 15, TAT: 12, WT: 7

Average TAT: 5.40 ms
Average WT: 2.40 ms
```

2. Write a C program to simulate SJF scheduling algorithm pre-emptive.

```
#include <stdio.h>
#define MAX 10

struct process {
   int at, bt, rt, ct, wt, tat;
};

void input(struct process p[], int n) {
   for(int i=0; i<n; i++) {
      printf("Enter arrival time and burst time of process %d: ", i+1);
      scanf("%d%d", &p[i].at, &p[i].bt);
      p[i].rt = p[i].bt;
   }
}

void srtf(struct process p[], int n) {
   int time = 0, completed = 0;
   float avgTat = 0, avgWt = 0;

   while(completed < n) {
      int idx = -1;
      int minRt = 9999;

      for(int i=0; i<n; i++) {
         if(p[i].at <= time && p[i].rt > 0) {
            if(p[i].rt < minRt) {
               minRt = p[i].rt;
               idx = i;
            }
         }
      }

      if(idx == -1) {
         time++;
         continue;
      }

      p[idx].rt--;
      time++;

      if(p[idx].rt == 0) {
         p[idx].ct = time;
         p[idx].tat = p[idx].ct - p[idx].at;
         p[idx].wt = p[idx].tat - p[idx].bt;
         completed++;
         printf("Process %d -> CT: %d, TAT: %d, WT: %d\n", idx+1, p[idx].ct, p[idx].tat,
```

```
p[idx].wt);
        }
    }

    for(int i=0; i<n; i++) {
        avgTat += p[i].tat;
        avgWt += p[i].wt;
    }

    printf("\nAverage TAT: %.2f ms\n", avgTat/n);
    printf("Average WT: %.2f ms\n", avgWt/n);
}

int main() {
    struct process p[MAX];
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    input(p, n);
    srtf(p, n);
    return 0;
}
```

Output:

```
Enter number of processes: 5
Enter arrival time and burst time of process 1: 0
3
Enter arrival time and burst time of process 2: 2
22
Enter arrival time and burst time of process 3: 3
5
Enter arrival time and burst time of process 4: 4
4
Enter arrival time and burst time of process 5: 6
1
Process 1 -> CT: 3, TAT: 3, WT: 0
Process 5 -> CT: 7, TAT: 1, WT: 0
Process 3 -> CT: 9, TAT: 6, WT: 1
Process 4 -> CT: 13, TAT: 9, WT: 5
Process 2 -> CT: 35, TAT: 33, WT: 11

Average TAT: 10.40 ms
Average WT: 3.40 ms
```

3. Write a C program to simulate priority scheduling algorithm non pre-emptive.

```
#include <stdio.h>

#define MAX 10


struct process {

   int at, bt, ct, wt, tat, prt;

};


void input(struct process p[], int n) {

   for(int i=0; i<n; i++) {

      printf("Enter AT, BT, Priority for Process %d: ", i+1);

      scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].prt);

   }

}


void priorityScheduling(struct process p[], int n) {

   int time = 0;

   float avgTat = 0, avgWt = 0;


   for(int i=0; i<n-1; i++) {

      for(int j=i+1; j<n; j++) {

         if(p[i].at > p[j].at ||

            (p[i].at == p[j].at && p[i].prt > p[j].prt)) {

             struct process temp = p[i];

             p[i] = p[j];

             p[j] = temp;

         }

      }

   }
```

```
    for(int i=0; i<n; i++) {
        if(time < p[i].at) {
            time = p[i].at;
        }
        time += p[i].bt;
        p[i].ct = time;
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;

        avgTat += p[i].tat;
        avgWt += p[i].wt;
    }

    printf("\nProcess\tAT\tBT\tPRT\tCT\tTAT\tWT\n");
    for(int i=0; i<n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               i+1, p[i].at, p[i].bt, p[i].prt,
               p[i].ct, p[i].tat, p[i].wt);
    }

    printf("\nAverage TAT: %.2f\n", avgTat/n);
    printf("Average WT: %.2f\n", avgWt/n);
}

int main() {
    struct process p[MAX];
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    input(p, n);
    priorityScheduling(p, n);
```

```
    return 0;
}
```

Output:

```
Enter number of processes: 5
Enter AT, BT, Priority for Process 1: 0
3
5
Enter AT, BT, Priority for Process 2: 2
2
3
Enter AT, BT, Priority for Process 3: 3
5
2
Enter AT, BT, Priority for Process 4: 4
4
4
Enter AT, BT, Priority for Process 5: 6
1
1

Process AT      BT      PRT     CT      TAT     WT
P1      0       3       5       3       3       0
P2      2       2       3       5       3       1
P3      3       5       2       10      7       2
P4      4       4       4       14      10      6
P5      6       1       1       15      9       8

Average TAT: 6.40
Average WT: 3.40
```

4. Write a C program to simulate priority scheduling algorithm preemptive.

```
#include <stdio.h>

#define MAX 20


struct process {

   int at, bt, rt, ct, wt, tat, prt;

};


void input(struct process p[], int n) {

   for(int i=0; i<n; i++) {

      printf("Enter AT, BT, Priority for Process %d: ", i+1);

      scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].prt);

      p[i].rt = p[i].bt;

   }

}


void priorityPreemptive(struct process p[], int n) {

   int completed = 0, time = 0;

   float avgTat = 0, avgWt = 0;


   while(completed < n) {

      int idx = -1;

      int highestPriority = 9999;


      for(int i=0; i<n; i++) {

         if(p[i].at <= time && p[i].rt > 0) {

            if(p[i].prt < highestPriority) {

               highestPriority = p[i].prt;

               idx = i;

            }

            else if(p[i].prt == highestPriority) {
```

```
            if(p[i].at < p[idx].at) {
                idx = i;
            }
        }
    }
  }

  if(idx != -1) {
    p[idx].rt--;
    time++;
    if(p[idx].rt == 0) {
        p[idx].ct = time;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;
        avgTat += p[idx].tat;
        avgWt += p[idx].wt;
        completed++;
    }
  } else {
    time++;
  }
}

printf("\nProcess\tAT\tBT\tPRT\tCT\tTAT\tWT\n");
for(int i=0; i<n; i++) {
  printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
      i+1, p[i].at, p[i].bt, p[i].prt,
      p[i].ct, p[i].tat, p[i].wt);
}

printf("\nAverage TAT: %.2f\n", avgTat/n);
```

```
    printf("Average WT: %.2f\n", avgWt/n);
}

int main() {
    struct process p[MAX];
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    input(p, n);
    priorityPreemptive(p, n);
    return 0;
}
```

Output:

```
Enter number of processes: 5
Enter AT, BT, Priority for Process 1: 0
3
5
Enter AT, BT, Priority for Process 2: 2
2
3
Enter AT, BT, Priority for Process 3: 3
5
2
Enter AT, BT, Priority for Process 4: 4
4
4
Enter AT, BT, Priority for Process 5: 6
1
1

Process AT      BT      PRT     CT      TAT     WT
P1      0       3       5       15      15      12
P2      2       2       3       10      8       6
P3      3       5       2       9       6       1
P4      4       4       4       14      10      6
P5      6       1       1       7       1       0

Average TAT: 8.00
Average WT: 5.00
```